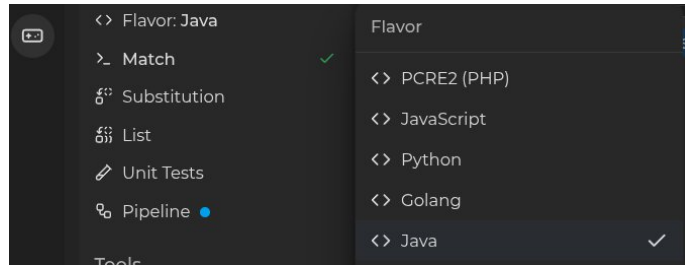


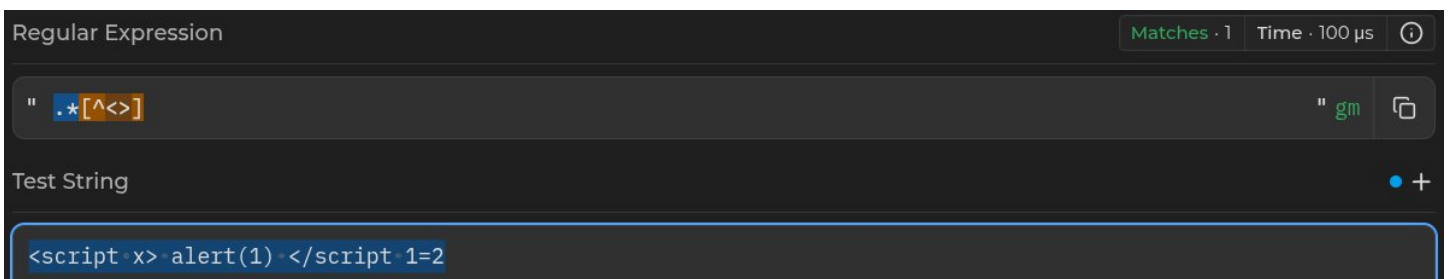
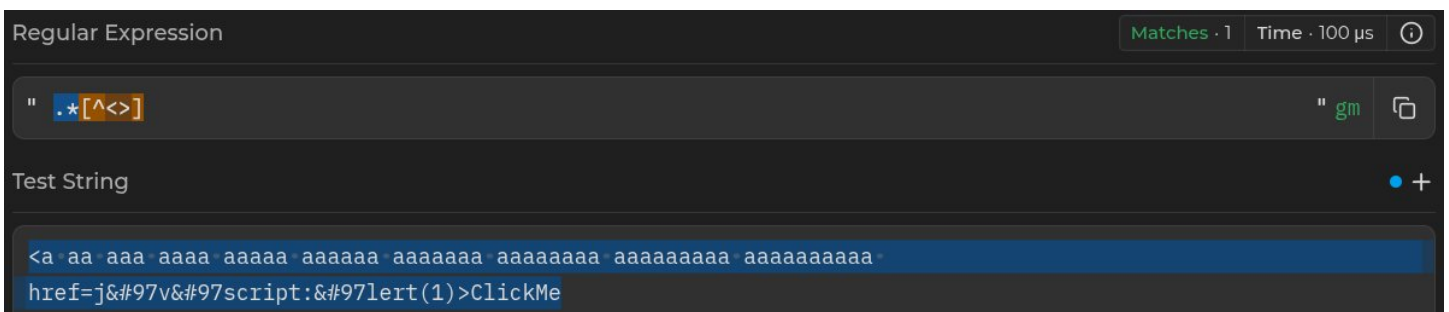
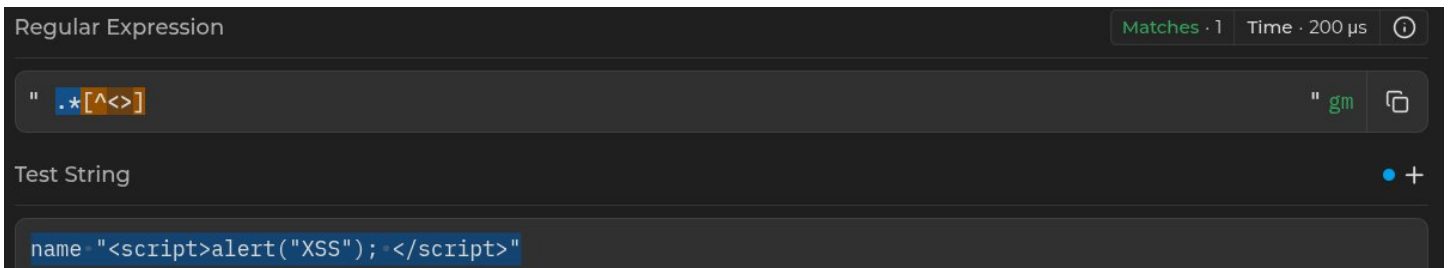
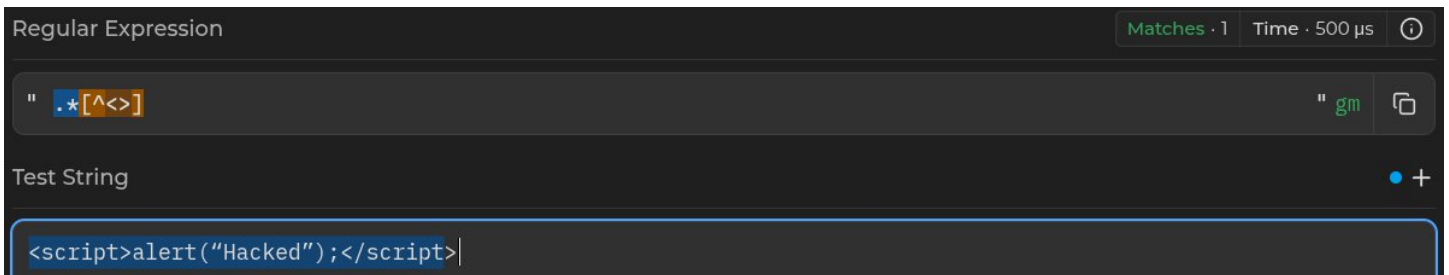
## Task 4.2C

I updated the flavor.



Regex: `.*[<>]`

Only checks the last character, so it can be bypassed if the payload does not end with < or >.



Regex: `[^<>].*`

Only checks the first character, so it can be bypassed if the payload does not start with `<` or `>`.

Regular Expression Matches · 1 Time · 100 µs ⓘ

" `[^<>].*` " gm

Test String • +

`<script>alert("Hacked");</script>`

Regular Expression Matches · 1 Time · 100 µs ⓘ

" `[^<>].*` " gm

Test String • +

`name="<script>alert("XSS");</script>"`

Regular Expression Matches · 1 Time · 0 µs ⓘ

" `[^<>].*` " gm

Test String • +

`<a aa aaa aaaa aaaaa aaaaaa aaaaaaa aaaaaaaaa aaaaaaaaaa aaaaaaaaaa href=j&#97v&#97script:&#97lert(1)>ClickMe`

Regular Expression Matches · 1 Time · 0 µs ⓘ

" `[^<>].*` " gm

Test String • +

`<script x> alert(1) </script 1=2`

Regex: `[^<>]+`

Checks the whole input and blocks it if `<` or `>` appears anywhere.

```
Regular Expression Matches · 1 Time · 0 μs ⓘ  
" [^<>].* " gm   
Test String • +  
<script>alert("Hacked");</script>
```

```
Regular Expression Matches · 1 Time · 100 μs ⓘ  
" [^<>].* " gm   
Test String • +  
name="<script>alert("XSS");</script>"
```

```
Regular Expression Matches · 1 Time · 0 μs ⓘ  
" [^<>].* " gm   
Test String • +  
<a aa aaa aaaa aaaaa aaaaaa aaaaaaa aaaaaaaaa aaaaaaaaaa aaaaaaaaaa href=j&#97v&#97script:&#97lert(1)>ClickMe
```

```
Regular Expression Matches · 1 Time · 0 μs ⓘ  
" [^<>].* " gm   
Test String • +  
<script x> alert(1) </script 1=2
```

Regex: **[a-zA-Z]+**

Only allows letters, so anything else is rejected. This is a whitelist approach.

```
Regular Expression Matches · 4 Time · 100 µs ⓘ  
" [a-zA-Z]+" gm   
Test String +  
<script>alert("Hacked");</script>
```

```
Regular Expression Matches · 5 Time · 100 µs ⓘ  
" [a-zA-Z]+" gm   
Test String +  
name="<script>alert("XSS");</script>
```

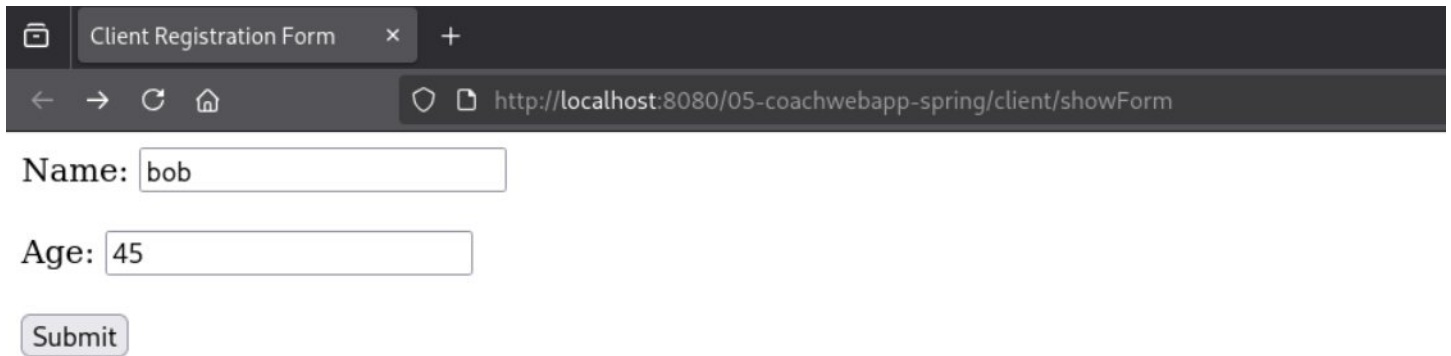
```
Regular Expression Matches · 16 Time · 100 µs ⓘ  
" [a-zA-Z]+" gm   
Test String +  
<a aa aaa aaaa aaaaa aaaaaa aaaaaaa aaaaaaaa aaaaaaaaa aaaaaaaaaa  
href=j&#97v&#97script:&#97lert(1)>ClickMe
```

```
Regular Expression Matches · 4 Time · 100 µs ⓘ  
" [a-zA-Z]+" gm   
Test String +  
<script>x> alert(1)</script>1=2
```

Therefore, this is the answer and this is what we will use for the rest of this task: **[a-zA-Z]+** I updated the source code as necessary.

```
Client.java x ClientController.java CoachController.java  
1 package edu.deakin.sit218.coachwebapp;  
2  
3 import javax.validation.constraints.Max;  
4 //import org.hibernate.validator.constraints  
5  
6 public class Client {  
7  
8     @NotNull(message = "is required")  
9     @Size(min = 3, message="is required")  
10    @Pattern(regexp = "[a-zA-Z]+", message="incorrect format")  
11    protected String name;  
12  
13  
14  
15
```

I ran the program with sample values:



Client Registration Form

← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/client/showForm

Name:

Age:



← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

# bob, please, run for 30 min

But testing with the sample XSS payloads fail as intended.

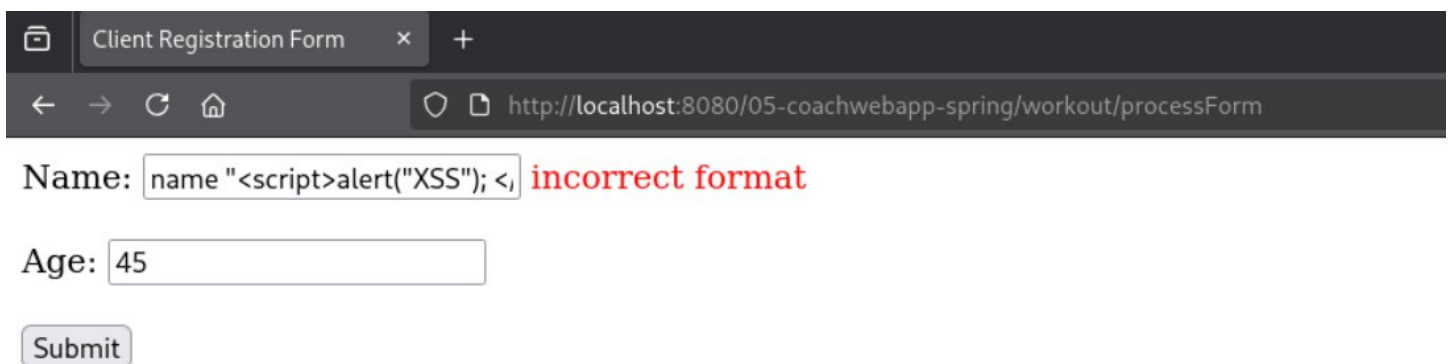


← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name:

Age:

Next, I tested out the other payloads as well.



Client Registration Form

← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name:

Age:

← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name: <a aa aaa aaaa aaaaa aaaaaa a incorrect format

Age: 45

Submit

← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name: <script x> alert(1) </script 1=2 incorrect format

Age: 45

Submit

However, another thing to note is that this regex will also reject legitimate names that has spaces, or apostrophes like O'Brien.

← → ↻ 🏠 http://localhost:8080/05-coachwebapp-spring/workout/processForm

Name: O'Brien incorrect format

Age: 45

Submit

I would slightly modify the regular expression to also allow spaces, dashes and apostrophes.

**[a-zA-Z\s'-.]+**

This would still exclude other dangerous characters like <, >, \$, etc...